CSD457 INTERNET OF THINGS · GROUP G-04

Poison-Resilient On-Device Intrusion Detection for Drifting IoT

Presentation notes and study guide

Rishit Kamboj and Farhan Naik · First presentation, 24 September 2026

Part	What is inside
1	The whole project on one page
2	Key ideas explained simply
3	The 5 papers: the crux of each, and links to read them
4	Our simulation: what it shows and how to explain it
5	Slide-by-slide presentation script with timing
6	Question and answer preparation
7	What happens after the presentation

How to use this: read Parts 1 to 3 to understand the project. Rehearse with Part 5. Each of you should be able to answer every question in Part 6 alone.

PART 1

The whole project on one page

> **In one sentence:** we are building an IoT intrusion detector that keeps learning on an ESP32 but cannot be taught to ignore attacks.

The problem in three steps

- **Detection should run on the device.** A small anomaly detector can sit on cheap hardware next to the traffic it protects [3].
- **But normal traffic keeps changing** (called *concept drift*). Real networks drift for weeks; traffic volume fell 52% at semester end in real ISP data [5]. So the detector must keep learning.
- **Learning is the weak point.** An attacker can feed it slow, harmless-looking changes until an attack looks normal. This is the *boiling-frog* poisoning attack [1]. The bigger the attacker's share of the traffic, the further they can push it [2].

What we will do

- **Measure the risk:** show how easily real, low-traffic IoT devices can be poisoned [2][4].
- **Build the defense, Threat-Orthogonal Adaptation (TOA):** split every update the detector wants to make. The part that moves sideways or away from known attacks is applied freely. Only the part that moves toward a known attack is rationed.
- **Prove it for real:** test on real IoT traffic [4] and real drift [5], then run it on an ESP32 and measure memory, speed and energy.

Why it is new

- The closest existing defense against boiling-frog poisoning (ANTIDOTE [1]) runs on a server with batch retraining. Ours learns continuously on a microcontroller.
- We found a **new attack**: defenses that *freeze* when drift looks suspicious can be tripped on purpose, leaving the detector stale and flooding operators with false alarms.
- TOA never freezes, and gives a **provable minimum time** before a known attack can be hidden.

Evidence so far

In our synthetic simulation with the attacker at 40% of traffic: naive learning loses the attack in 24 cycles; freezing defenses keep the attack visible but reach 85 to 87% false alarms; TOA keeps 100% of the attack detected with 0.3% false alarms. Real-data results come by mid-review.

PART 2

Key ideas explained simply

Term	Plain meaning	Why it matters here
Intrusion detection system (IDS)	Software that watches network traffic and raises an alarm on attacks.	What we are building, on an ESP32.
Anomaly detection	Learn what normal looks like; alarm on anything too different. Can catch unseen attacks.	Our detector type [3]. It needs a model of normal, which is what an attacker targets.
Centre and spread	Our detector keeps the average of normal traffic (centre) and how spread out it is. Far from the centre means alarm.	Both can be poisoned: drag the centre, or inflate the spread.
Concept drift	Normal behaviour slowly changes: new devices, firmware, seasons, semesters.	Forces the detector to keep learning [5].
Data poisoning	Feeding a learning system crafted data so it learns the wrong thing.	The attack we defend against.
Boiling-frog attack	Poison in tiny steps, each too small to alarm, until an attack looks normal.	Demonstrated on network detectors in [1].
Attacker's share	What fraction of the learning data the attacker controls.	Bounds how far they can push [2]; small IoT devices make it large.
Freeze attack (ours)	Trip a defense that stops learning, then leave; drift makes the frozen detector useless.	Our new finding; TOA is immune because it never freezes.
Threat-Orthogonal Adaptation	Split each update into toward-attack and everything else; only ration toward-attack.	Our method.
Margin	Distance between a known attack and the edge of what counts as normal.	TOA guarantees it shrinks slowly, which gives the proof.
Recall / false-positive rate	Share of attacks caught / share of normal traffic wrongly flagged.	Our two headline numbers.

The key intuition behind TOA

Honest drift can move in any direction. A semester-end traffic drop, for example, moves away from flood attacks. Poisoning has no such freedom: to hide an attack it **must** move normal toward that attack. So instead of blocking all drift (which freezes the detector), we block only drift in the one direction an attacker needs. On an ESP32 that costs one dot product per known attack per update.

PART 3

The 5 papers: the crux of each

Chosen from 78 papers found and 28 read in full. Each one gives us one piece; together they show the gap. Read each 'Start with' section before the presentation.

[1] ANTIDOTE: Understanding and Defending against Poisoning of Anomaly Detectors

B. I. P. Rubinstein, B. Nelson, L. Huang, A. D. Joseph et al. · ACM Internet Measurement Conference (IMC), 2009

Question it asks	Network operators use a PCA-based detector to spot anomalies in backbone traffic, and they retrain it on recent traffic. Can an attacker poison that retraining?
The crux	Yes. The paper introduces the **boiling-frog** strategy: add a little malicious traffic ('chaff') over several weeks so the detector slowly accepts it, then launch the real attack unnoticed. As a defense it proposes **ANTIDOTE**, a robust version of PCA (PCA-GRID) that is less influenced by outliers, and shows it sharply reduces the attacker's success.
Why it matters to us	It proves our threat is real on network detectors, and it is the closest existing defense, which we cite honestly as prior work.
What it leaves open	Server-side, batch retraining on large backbone traffic. Not continuous learning, not on a small device, and it does not study defenses that freeze.
Start with	Abstract, the Boiling Frog section, and the ANTIDOTE results.
Read it	https://people.eecs.berkeley.edu/~tygar/papers/SML/IMC.2009.pdf

[2] Security Analysis of Online Centroid Anomaly Detection

M. Kloft and P. Laskov · Journal of Machine Learning Research (JMLR), vol. 13, 2012

Question it asks	An online centroid detector keeps updating its centre as new data arrives. How much can an attacker move it?
The crux	A formal analysis: it derives the optimal attack and proves that how far the attacker can shift the centre is **bounded by the fraction of the traffic they control**. With a small share, the attack stalls; with a large share, it succeeds.
Why it matters to us	This is the theory behind our finding that small IoT devices are easier to poison: a smart plug sends little traffic, so the attacker's share can be large. Our simulation matches it: at 10 to 20% share even naive learning survives; at 30%+ it breaks.
What it leaves open	Pure theory. Nobody has measured attacker share on real low-traffic IoT devices, or built an on-device defense from it.
Start with	Introduction and the results on attack efficiency versus attacker fraction.
Read it	https://jmlr.csail.mit.edu/papers/volume13/kloft12b/kloft12b.pdf Also: https://arxiv.org/abs/1003.0078

[3] Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection

Y. Mirsky, T. Doitshman, Y. Elovici and A. Shabtai · NDSS, 2018

Question it asks	Can an intrusion detector learn normal traffic by itself, online, cheaply enough for small devices?
The crux	Kitsune extracts compact statistics from each packet stream in constant memory, then scores anomalies with a small ensemble of autoencoders. It runs on a Raspberry Pi and detects real attacks, including the Mirai botnet.
Why it matters to us	Source of our feature extraction, and proof that on-device anomaly detection is practical. Serves as our static baseline.
What it leaves open	After its training phase it stops learning, so it goes stale under drift, and it has no protection against poisoning.
Start with	Sections on the feature extractor (why it is cheap) and the evaluation.
Read it	https://arxiv.org/abs/1802.09089

[4] N-BaIoT: Network-Based Detection of IoT Botnet Attacks Using Deep Autoencoders

Y. Meidan, M. Bohadana, Y. Mathov et al. · IEEE Pervasive Computing, 2018

Question it asks	How do real IoT devices behave on the network when infected by botnets?
The crux	Captured traffic from **9 real commercial IoT devices** (cameras, doorbells, thermostats, a baby monitor) in normal operation and while infected with **Mirai and BASHLITE**. Trained one autoencoder per device and detected the attacks accurately.
Why it matters to us	Our main dataset: real IoT traffic and real attacks, with realistic per-device traffic rates that tell us how large an attacker's share could be.
What it leaves open	Detection is offline; no drift over time and no poisoning of the detector.
Start with	The dataset description and the list of devices and attacks.
Read it	https://arxiv.org/abs/1805.03409

[5] CESNET-TimeSeries24: Time Series Dataset for Network Traffic Anomaly Detection and Forecasting

J. Koumar, K. Hynek, T. Čejka and P. Šiška · Scientific Data, vol. 12, 2025

Question it asks	Researchers lack long, real network traffic datasets to study how traffic changes over time.
The crux	Released **40 weeks** of traffic statistics from an ISP network serving about half a million users: 275,000 IP addresses, 66 billion flows, at IP, institution and subnet level.
Why it matters to us	Our source of **real benign drift**. It showed us that honest drift is steady and directional too (volume down 52% at semester end, from an analysis of this data), which is why direction alone cannot reveal an attack.
What it leaves open	A dataset. It has not been used to test poisoning defenses for adaptive detectors.
Start with	The dataset overview and the figures of traffic over time.
Read it	https://www.nature.com/articles/s41597-025-04603-x Also: https://arxiv.org/abs/2409.18874

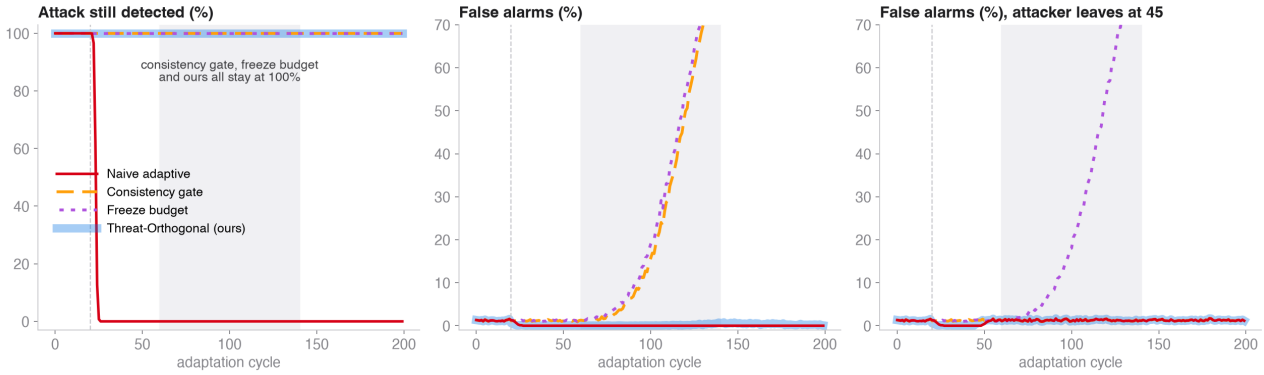
How the five fit together

Kitsune [3] shows detection can live on a device, but it stops learning. CESNET-TS24 [5] shows real traffic drifts, so it must keep learning. ANTIDOTE [1] shows learning can be poisoned and defends only on a server. Kloft and Laskov [2] explain when poisoning succeeds, and N-BaIoT [4] gives real IoT devices to test that on. The missing piece, a detector that learns safely on the device, is our project.

PART 4

Our simulation: what it shows

We tested the idea before presenting it, in **experiments/sim_toa.py** (synthetic data, 5 random seeds). A centre-and-spread detector learns each cycle from traffic it judges normal. The attacker controls 40% of traffic and places poison just inside the boundary, toward the attack it wants to hide. Honest drift happens during cycles 60 to 140 (grey band).



Policy	Attack still detected	False alarms	What happened
Naive learning	0% after cycle 24	0%	Poisoned: the attack now looks normal.
Robust consistency filter	100%	85%	Stops learning, then drowns in false alarms once drift starts.
Freeze budget	100%	87%	Freezes. In the right panel the attacker leaves at cycle 45 and it never recovers: the freeze attack.
TOA (ours)	100%	0.3%	Keeps learning honest drift; the attacker cannot move it toward the attack.

Honest limits to say out loud

- The chart is synthetic, but a real check on N-BaIoT doorbell traffic agrees: naive learning poisoned at 10 to 20% share (10% for scan, 20% for ack and syn), ours 100% detection with 0.8% false alarms (experiments/nbaiot_check.py).
- The slide version uses attack signatures. A signature-free version also works on real N-BaIoT (both attackers, 100% detection), but an attack hidden inside strong genuine drift still beats it in synthetic tests.
- If honest drift moves toward an attack, TOA slows learning in that direction; false alarms peaked near 7% in our worst-case test, then recovered.
- At a 10 to 20% attacker share even naive learning survived, consistent with Kloft and Laskov [2].

PART 5

Slide-by-slide script

About 7 minutes of talking, leaving 3 for questions. The speaker split is a suggestion; swap slides so each of you presents the part you understand best. Speak naturally; do not read these word for word.

#	Slide	Time	Speaker	What to say
1	Title	20 s	Rishit	We are Group G-04. Our project is about intrusion detection that runs on a small IoT device and keeps learning, without an attacker being able to abuse that learning.
2	What we will research	40 s	Rishit	In one line: an IoT intrusion detector that keeps learning on an ESP32 and cannot be taught to ignore attacks. We will do three things: measure how easily real IoT devices get poisoned, build our defense called Threat-Orthogonal Adaptation, and prove it on real data and real hardware.
3	Problem	45 s	Rishit	Detection belongs on the device [3]. But normal traffic keeps changing, as real data shows [5], so the detector must keep learning. And that learning is the weak point: an attacker can feed slow, harmless-looking changes until an attack looks normal [1], and the bigger their share of traffic, the easier it is [2].
4	The 5 papers	70 s	Rishit	We read 28 papers in full and built on these five. Walk the rows, and read the red column: server only, theory only, no defense, no drift, never used for this. That red column is our research gap.
5	Research gap	40 s	Farhan	Detectors on devices stop learning, real drift means learning cannot stop, and the one existing defense runs on a server. So nothing learns safely on a device. Credit where due: the attack and a server defense both come from ANTIDOTE.
6	Research questions	35 s	Farhan	RQ1 is our method. RQ2 is a new attack we found. RQ3 is real exposure and cost on the ESP32.
7	Approach	60 s	Farhan	Every update the detector wants to make is split in two. Anything sideways or away from known attacks is applied immediately, so it never freezes. Only the part moving toward an attack is rationed. That gives a provable minimum time before an attack can be hidden, for the cost of a few dot products.
8	Three findings	50 s	Farhan	One: honest drift is directional too, so direction toward an attack is what matters. Two: defenses that freeze can be weaponized, which broke our own first design. Three: small IoT devices are easier to poison because the attacker can be a large share of their traffic.
9	Platform and baselines	40 s	Rishit	ESP32 hardware, Python for experiments, N-BaIoT for real IoT attacks and CESNET for real drift. We compare against five baselines, including the classic ANTIDOTE defense.
10	Preliminary result	60 s	Rishit	This is synthetic, to test the idea. Left: naive learning is poisoned in 24 cycles. Middle: freezing defenses keep the attack visible but false alarms explode once drift starts. Right: the attacker leaves at cycle 45 and the frozen defense never recovers. Blue is ours in every panel.
11	Contribution and plan	40 s	Farhan	Three contributions: the method, the freeze attack, and real exposure and cost measurements. Real-data results by mid-review, the ESP32 port after that. Thank you, we are happy to take questions.

#	Slide	Time	Speaker	What to say
12	References	Q&A;	Both	Leave this slide up during questions.

Delivery tips

- Open with slide 2's sentence. If the audience remembers one thing, it should be that.
- Point at the red column on slide 4 and the blue line on slide 10. Those two moments carry the talk.
- Say 'synthetic' clearly on slide 10. Honesty about limits earns more credit than overclaiming.
- Credit ANTIDOTE out loud on slide 5. It shows you read deeply and makes your novelty claim safe.
- Rehearse once with a timer. If you run long, shorten slide 4 first.

PART 6

Question and answer preparation

Isn't this just ANTIDOTE?

No. ANTIDOTE [1] retrains a robust PCA model in batches on a server with lots of traffic. We learn continuously on a microcontroller, never freeze, and handle drift. ANTIDOTE is one of our baselines.

Is the boiling-frog attack your idea?

No, it is from ANTIDOTE [1]. Our contributions are the defense (TOA), the freeze attack, and measurements on real IoT data and hardware.

What is the freeze attack exactly?

Many defenses stop learning when drift looks suspicious. An attacker can trigger that on purpose and leave. Normal traffic keeps drifting, so the frozen detector raises more and more false alarms until someone switches it off. TOA never freezes, so it cannot be tricked this way.

Why would an attacker control 40% of traffic?

Kloft and Laskov [2] show poisoning only works with a large share. Small IoT devices like smart plugs send very little traffic [4], so an attacker easily matches it. At 10 to 20% even naive learning survived in our simulation, which agrees with [2].

Doesn't TOA need attack signatures? That sounds like hard-coding.

The version on the slides uses signatures for known attacks. We also built a signature-free version: it learns only from the dense core of traffic, and accepts a shift or a wider boundary only if the whole traffic cloud backs it up, because honest drift moves everything while poison moves one part. On real N-BaIoT doorbell traffic it kept 100% detection of Mirai scan, ack and syn against both a normal attacker and a smart attacker who knows the defense, at 10 to 50% attacker share, with 0.2 to 1.5% false alarms.

So what is still unsolved?

An attack hidden inside strong genuine drift. In synthetic tests where heavy honest drift and the attack coincide, the signature-free version was still poisoned. Our plan combines both: signature-free by default, with signatures as a guaranteed floor for the most important attacks such as Mirai. Closing that gap is the research question for the rest of the semester.

What if normal traffic drifts toward an attack?

Then TOA slows learning in that direction. In our worst-case test false alarms peaked near 7% and recovered, far better than freezing defenses.

Is the simulation real data?

The chart is synthetic. We also ran a real check on N-BaIoT: real doorbell traffic and real Mirai attacks, with a modelled poisoner. Naive learning was poisoned at just 10 to 20% attacker share (10% for scan, 20% for ack and syn); ours kept 100% detection with 0.8% false alarms across three Mirai attacks. Full real-data comparison comes by mid-review.

Why an ESP32?

It is cheap, common in IoT, and small enough that memory and energy really matter. That is exactly where continuous, safe learning is hardest and where no existing defense runs.

What is the guarantee?

TOA only lets the gap between normal and a known attack shrink by a fixed budget per cycle, so the attack stays detected for at least a computable number of cycles, whatever the attacker does.

What does it cost on the ESP32?

One dot product and one distance per known attack per update. Measuring the real memory, latency and energy is part of RQ3.

PART 7

What happens after the presentation

When	Goal	Output
Weeks 1 to 3	N-BaIoT pipeline with Kitsune-style features; naive, static and freeze baselines; the boiling-frog attack	Naive detector poisoned on real data
Week 4 (mid-review)	TOA on real data; literature table; architecture	Real-data version of slide 10
Weeks 5 to 7	ESP32 port; freeze attack study; CESNET drift replay	Memory, latency, energy numbers
Weeks 8 to 9	Sweeps over attacker share and budget; IEEE report; demo	Final report and code

Files in our repository

- **topic-proposal.md**: the full proposal.
- **research-findings.md**: why the design changed and the evidence.
- **literature-review.md**: all 28 papers read, with notes.
- **experiments/sim_toa.py**: the simulation (run: python3 sim_toa.py).
- Repository: https://github.com/IOT-snu/brainstorming

All five papers, one list

[1] ANTIDOTE: Understanding and Defending against Poisoning of Anomaly Detectors. B. I. P. Rubinstein, B. Nelson, L. Huang, A. D. Joseph et al. · ACM Internet Measurement Conference (IMC), 2009.
https://people.eecs.berkeley.edu/~tygar/papers/SML/IMC.2009.pdf

[2] Security Analysis of Online Centroid Anomaly Detection. M. Kloft and P. Laskov · Journal of Machine Learning Research (JMLR), vol. 13, 2012.
https://jmlr.csail.mit.edu/papers/volume13/kloft12b/kloft12b.pdf
https://arxiv.org/abs/1003.0078

[3] Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection. Y. Mirsky, T. Doitshman, Y. Elovici and A. Shabtai · NDSS, 2018.
https://arxiv.org/abs/1802.09089

[4] N-BaIoT: Network-Based Detection of IoT Botnet Attacks Using Deep Autoencoders. Y. Meidan, M. Bohadana, Y. Mathov et al. · IEEE Pervasive Computing, 2018.
https://arxiv.org/abs/1805.03409

[5] CESNET-TimeSeries24: Time Series Dataset for Network Traffic Anomaly Detection and Forecasting. J. Koumar, K. Hynek, T. Čejka and P. Šiška · Scientific Data, vol. 12, 2025.
https://www.nature.com/articles/s41597-025-04603-x
https://arxiv.org/abs/2409.18874